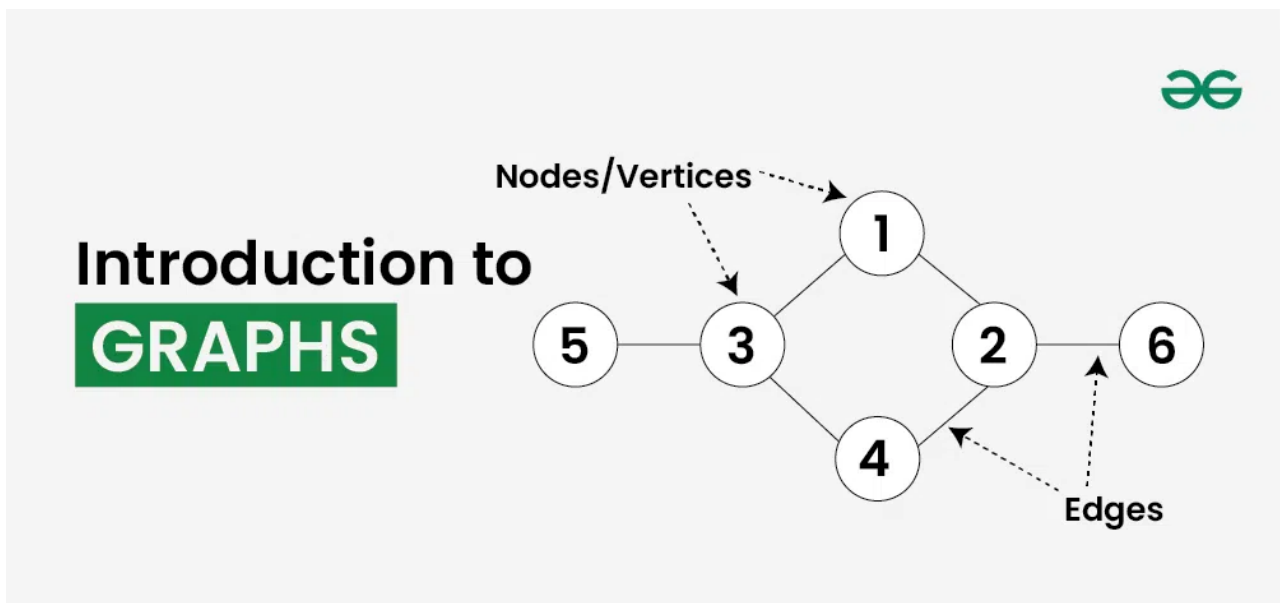


Introduction to Graph Data Structure

Last Updated : 12 Dec, 2024



Graph Data Structure is a [non-linear data structure](#) consisting of vertices and edges. It is useful in fields such as social network analysis, recommendation systems, and computer networks. In the field of sports data science, graph data structure can be used to analyze and understand the dynamics of team performance and player interactions on the field.



What is Graph Data Structure?

Graph is a [non-linear data structure](#) consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by $G(V, E)$.

Imagine a game of football as a web of connections, where players are the nodes and their interactions on the field are the edges. This web of connections is exactly what a graph data structure represents, and it's the key to unlocking insights into team performance and player dynamics in sports.

Components of Graph Data Structure

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.

- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labelled/unlabelled.

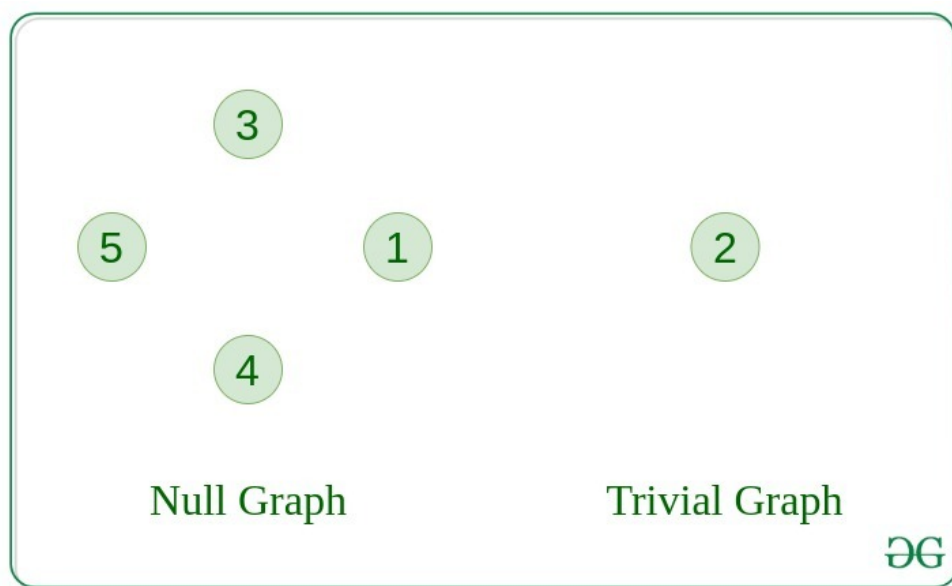
Types Of Graphs in Data Structure and Algorithms

1. Null Graph

A graph is known as a null graph if there are no edges in the graph.

2. Trivial Graph

Graph having only a single vertex, it is also the smallest graph possible.

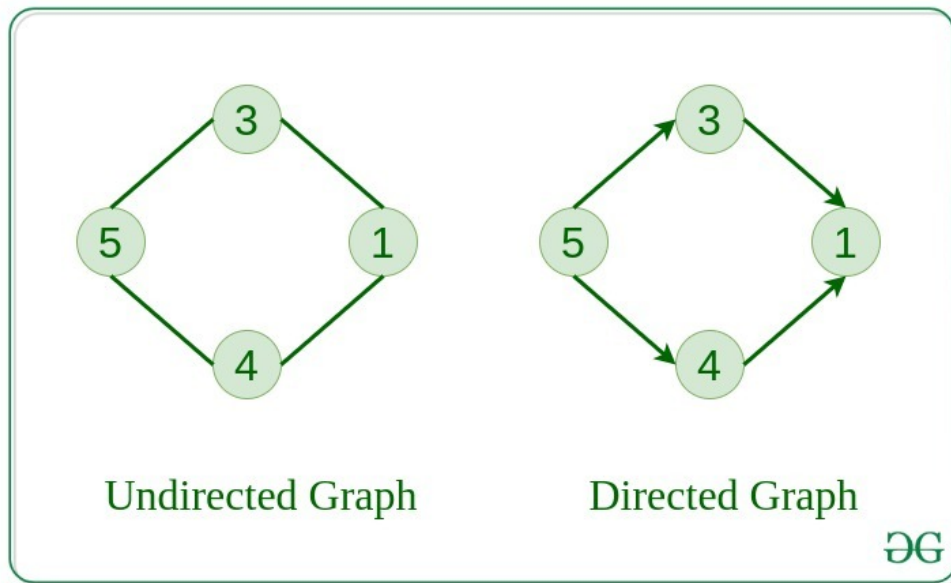


3. Undirected Graph

A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

4. Directed Graph

A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.

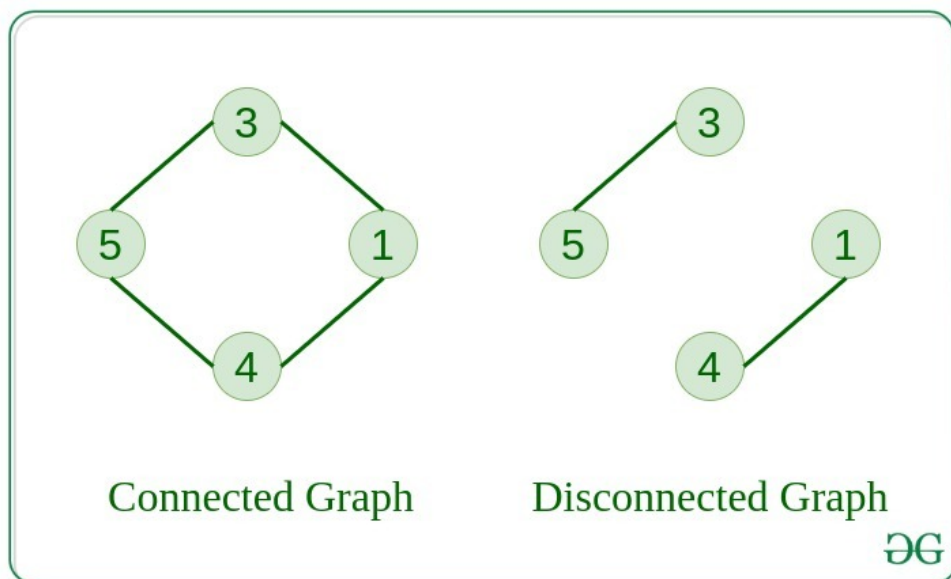


5. Connected Graph

The graph in which from one node we can visit any other node in the graph is known as a connected graph.

6. Disconnected Graph

The graph in which at least one node is not reachable from a node is known as a disconnected graph.

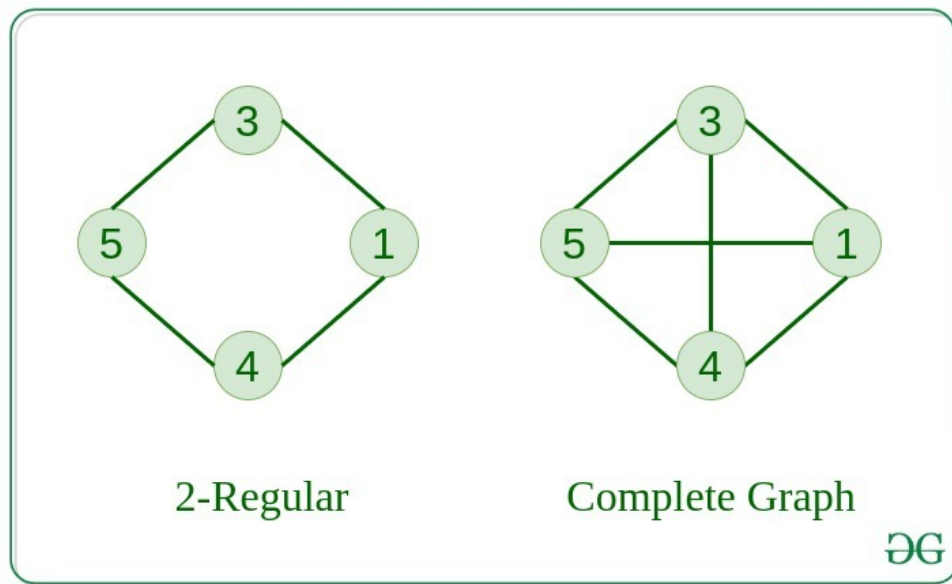


7. Regular Graph

The graph in which the degree of every vertex is equal to K is called K regular graph.

8. Complete Graph

The graph in which from each node there is an edge to each other node.

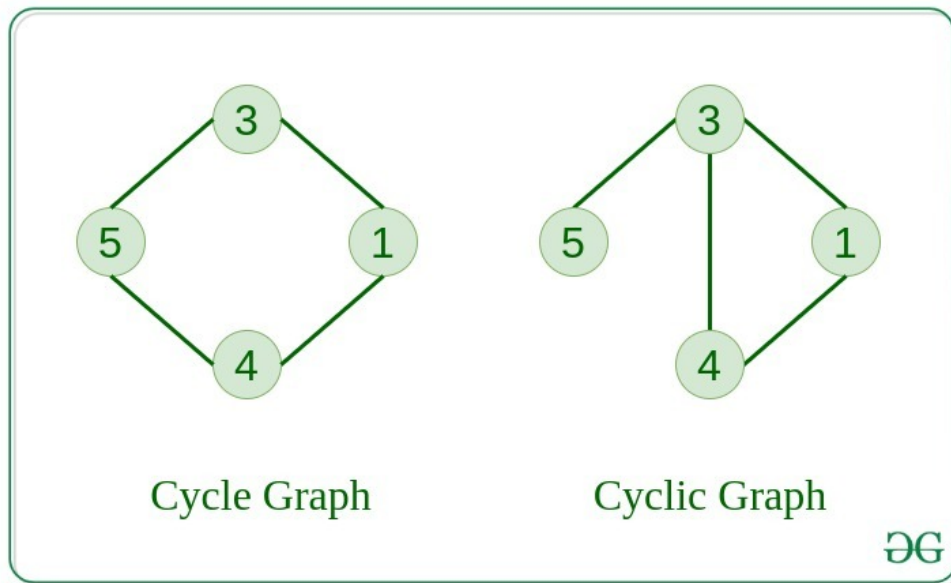


9. Cycle Graph

The graph in which the graph is a cycle in itself, the minimum value of degree of each vertex is 2.

10. Cyclic Graph

A graph containing at least one cycle is known as a Cyclic graph.

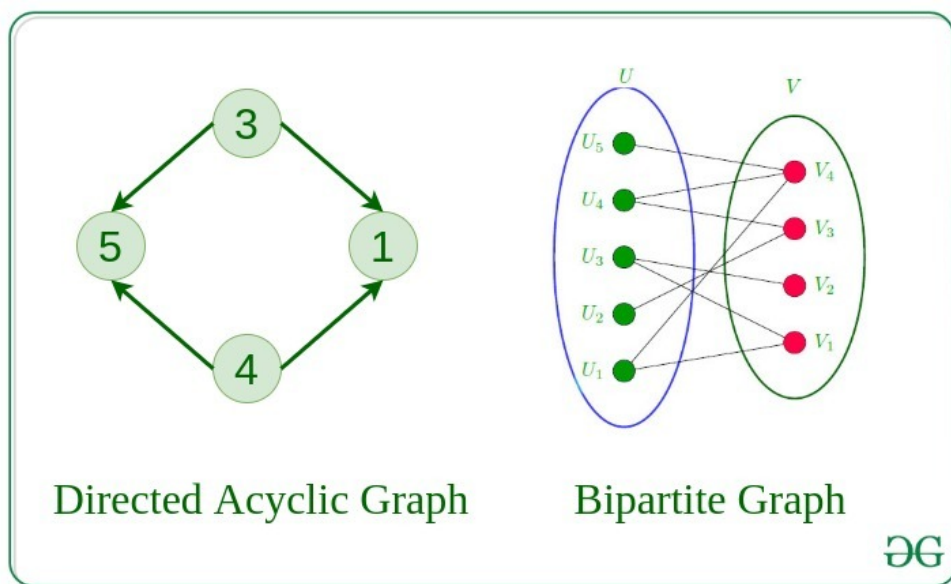


11. Directed Acyclic Graph

A Directed Graph that does not contain any cycle.

12. Bipartite Graph

A graph in which vertex can be divided into two sets such that vertex in each set does not contain any edge between them.



13. Weighted Graph

- A graph in which the edges are already specified with suitable weight is known as a weighted graph.

- Weighted graphs can be further classified as directed weighted graphs and undirected weighted graphs.

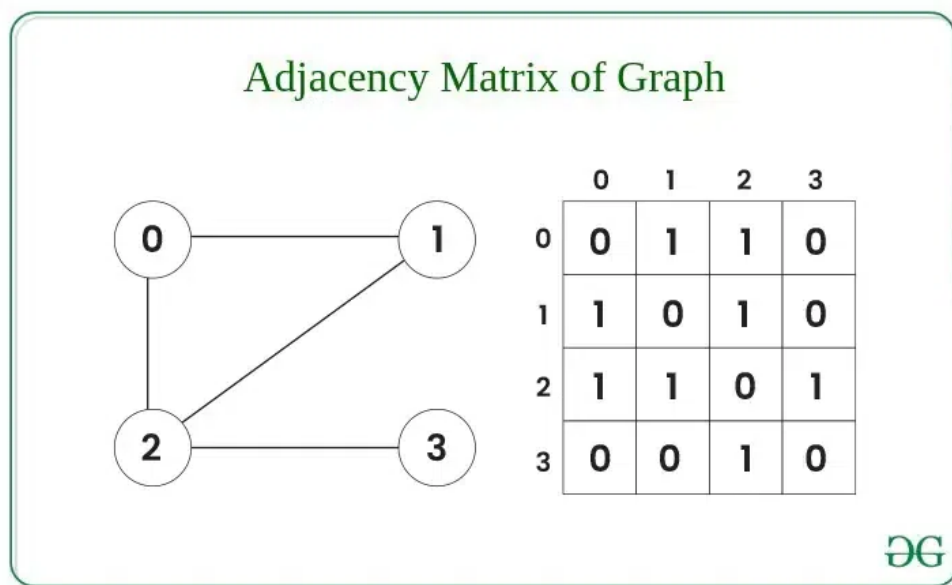
Representation of Graph Data Structure:

There are multiple ways to store a graph: The following are the most common representations.

- Adjacency Matrix
- Adjacency List

Adjacency Matrix Representation of Graph Data Structure:

In this method, the graph is stored in the form of the 2D matrix where rows and columns denote vertices. Each entry in the matrix represents the weight of the edge between those vertices.



Below is the implementation of Graph Data Structure represented using Adjacency Matrix:

C++ C Java Python C# JavaScript

```
// C++ program to demonstrate Adjacency Matrix
// representation of undirected and unweighted graph
#include <bits/stdc++.h>
using namespace std;
```

```

void addEdge(vector<vector<int>> &mat, int i, int j)
{
    mat[i][j] = 1;
    mat[j][i] = 1; // Since the graph is undirected
}

void displayMatrix(vector<vector<int>> &mat)
{
    int V = mat.size();
    for (int i = 0; i < V; i++)
    {
        for (int j = 0; j < V; j++)
            cout << mat[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    // Create a graph with 4 vertices and no edges
    // Note that all values are initialized as 0
    int V = 4;
    vector<vector<int>> mat(V, vector<int>(V, 0));

    // Now add edges one by one
    addEdge(mat, 0, 1);
    addEdge(mat, 0, 2);
    addEdge(mat, 1, 2);
    addEdge(mat, 2, 3);

    /* Alternatively we can also create using below
       code if we know all edges in advacem

    vector<vector<int>> mat = {{ 0, 1, 0, 0 },
                               { 1, 0, 1, 0 },
                               { 0, 1, 0, 1 },
                               { 0, 0, 1, 0 } }; */

    cout << "Adjacency Matrix Representation" << endl;
    displayMatrix(mat);
}

```

```
    return 0;  
}
```

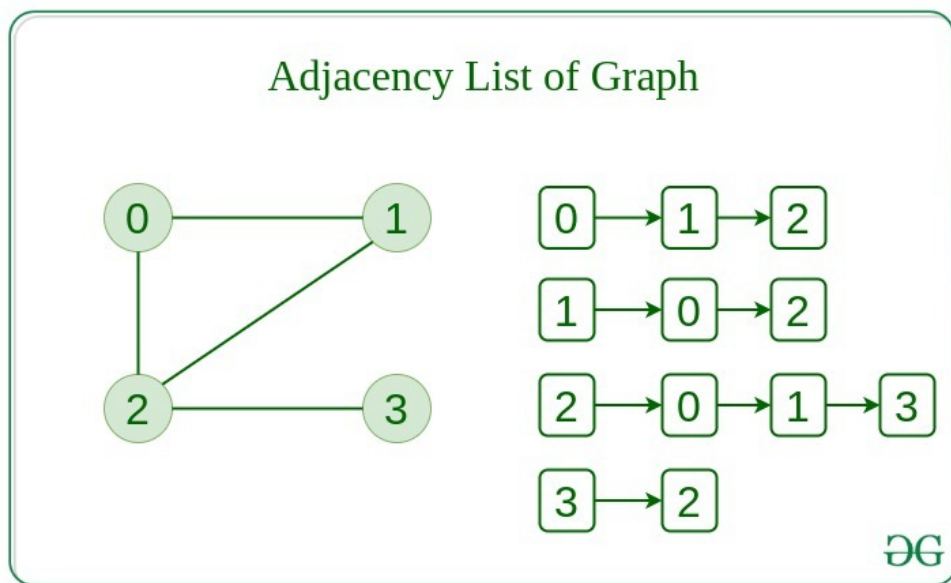
Output

Adjacency Matrix Representation

```
0 1 1 0  
1 0 1 0  
1 1 0 1  
0 0 1 0
```

Adjacency List Representation of Graph:

This graph is represented as a collection of linked lists. There is an array of pointer which points to the edges connected to that vertex.



Below is the implementation of Graph Data Structure represented using Adjacency List:

C++ C Java Python C# JavaScript

```
#include <iostream>  
#include <vector>  
using namespace std;  
  
// Function to add an edge between two vertices  
void addEdge(vector<vector<int>>& adj, int i, int j) {
```



```

adj[i].push_back(j);
adj[j].push_back(i); // Undirected
}

// Function to display the adjacency list
void displayAdjList(const vector<vector<int>>& adj) {
    for (int i = 0; i < adj.size(); i++) {
        cout << i << ": "; // Print the vertex
        for (int j : adj[i]) {
            cout << j << " "; // Print its adjacent
        }
        cout << endl;
    }
}

// Main function
int main() {
    // Create a graph with 4 vertices and no edges
    int V = 4;
    vector<vector<int>> adj(V);

    // Now add edges one by one
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 2);
    addEdge(adj, 1, 2);
    addEdge(adj, 2, 3);

    cout << "Adjacency List Representation:" << endl;
    displayAdjList(adj);

    return 0;
}

```

Output

```

Adjacency List Representation:
0: 1 2
1: 0 2
2: 0 1 3
3: 2

```

Comparison between Adjacency Matrix and Adjacency List

When the graph contains a large number of edges then it is good to store it as a matrix because only some entries in the matrix will be empty. An algorithm such as [Prim's](#) and [Dijkstra](#) adjacency matrix is used to have less complexity.

Action	Adjacency Matrix	Adjacency List
Adding Edge	$O(1)$	$O(1)$
Removing an edge	$O(1)$	$O(N)$
Initializing	$O(N*N)$	$O(N)$

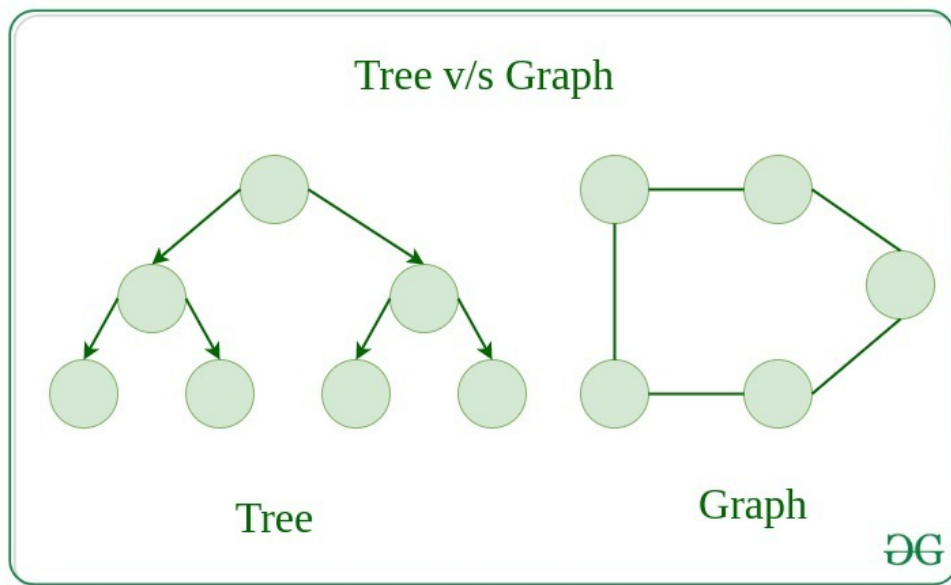
Basic Operations on Graph Data Structure:

Below are the basic operations on the graph:

- Insertion or Deletion of Nodes in the graph
 - [Add and Remove vertex in Adjacency List representation of Graph](#)
 - [Add and Remove vertex in Adjacency Matrix representation of Graph](#)
- Insertion or Deletion of Edges in the graph
 - [Add and Remove Edge in Adjacency List representation of a Graph](#)
 - [Add and Remove Edge in Adjacency Matrix representation of a Graph](#)
- Searching in Graph Data Structure- Search an entity in the graph.
- Traversal of Graph Data Structure- Traversing all the nodes in the graph.

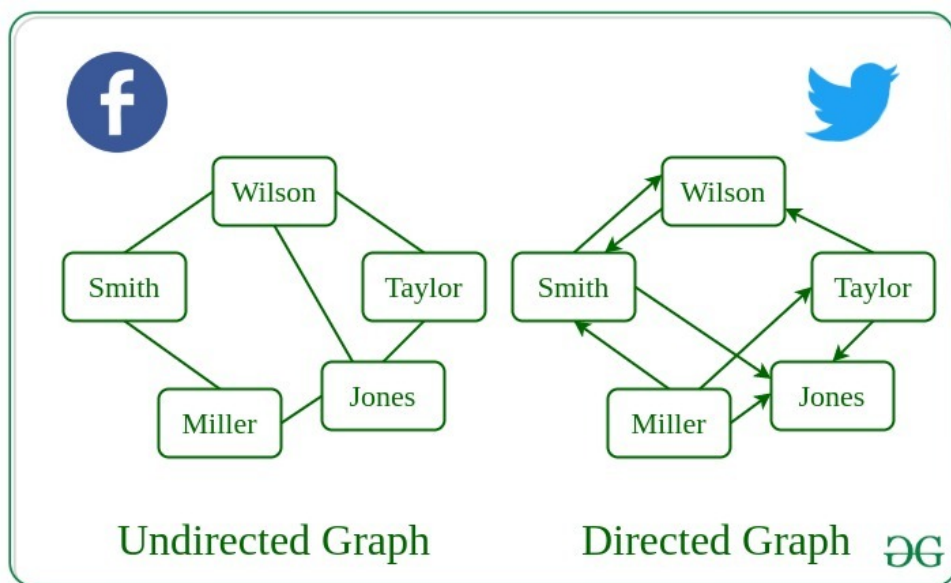
Difference between Tree and Graph:

Tree is a restricted type of Graph Data Structure, just with some more rules. Every tree will always be a graph but not all graphs will be trees. [Linked List](#), [Trees](#), and [Heaps](#) all are special cases of graphs.



Real-Life Applications of Graph Data Structure:

Graph Data Structure has numerous real-life applications across various fields. Some of them are listed below:



- If we recall all the previous data structures that we have studied like array, linked list, tree, etc. All these had some restrictions on structure (mostly linear and tree hierarchical which means no loops). Graph allows random connections between nodes which is useful in many real world problems where do have restrictions of previous data structures.
- Used heavily in social networks. Everyone on the network is a vertex (or node) of the graph and if connected, then there is an edge. Now imagine all the features that you see, mutual friends, people that follow you, etc can seen as graph problems.

- Used to represent the topology of computer networks, such as the connections between routers and switches.
- Used to represent the connections between different places in a transportation network, such as roads and airports.
- **Neural Networks:** Vertices represent neurons and edges represent the synapses between them. Neural networks are used to understand how our brain works and how connections change when we learn. The human brain has about 10^{11} neurons and close to 10^{15} synapses.
- **Compilers:** Graph Data Structure is used extensively in compilers. They can be used for type inference, for so-called data flow analysis, register allocation, and many other purposes. They are also used in specialized compilers, such as query optimization in database languages.
- **Robot planning:** Vertices represent states the robot can be in and the edges the possible transitions between the states. Such graph plans are used, for example, in planning paths for autonomous vehicles.
- Dependencies in a software project (or any other type of project) can be seen as graph and generating a sequence to solve all tasks before dependents is a standard graph topological sorting algorithm.
- For optimizing the cost of connecting all locations of a network. For example, minimizing wire length in a wired network to make sure all devices are connected is a standard Graph problem called Minimum Spanning Tree.

Advantages of Graph Data Structure:

- Graph Data Structure used to represent a wide range of relationships as we do not have any restrictions like previous data structures (Tree cannot have loops and have to be hierarchical. Arrays, Linked List, etc are linear)
- They can be used to model and solve a wide range of problems, including pathfinding, data clustering, network analysis, and machine learning.
- Any real world problem where we certain set of items and relations between them can be easily modeled as a graph and a lot of standard graph algorithms like BFS, DFS, Spanning Tree, Shortest Path, Topological Sorting and Strongly Connected
- Graph Data Structure can be used to represent complex data structures in a simple and intuitive way, making them easier to understand and analyze.

Disadvantages of Graph Data Structure:

- Graph Data Structure can be complex and difficult to understand, especially for people who are not familiar with graph theory or related algorithms.
- Creating and manipulating graphs can be computationally expensive, especially for very large or complex graphs.
- Graph algorithms can be difficult to design and implement correctly, and can be prone to bugs and errors.
- Graph Data Structure can be difficult to visualize and analyze, especially for very large or complex graphs, which can make it challenging to extract meaningful insights from the data.

More Resources of Graph:

- [Recent Articles on Graph](#)
- [Practice problems on Graph](#)
- [Algorithms on Graphs](#)

Introduction to Graph

[Visit Course ↗](#)[Comment](#)[More info ▼](#)[Advertise with us](#)[Next Article >](#)[Graph and its representations](#)

Similar Reads

Graph Algorithms

Graph algorithms are methods used to manipulate and analyze graphs, solving various range of problems like finding the shortest path, cycles detection. If you are looking for difficulty-wise list of problems,...

🕒 3 min read

Introduction to Graph Data Structure

Graph Data Structure is a non-linear data structure consisting of vertices and edges. It is useful in fields such as social network analysis, recommendation systems, and computer networks. In the field of sports...

🕒 15+ min read

Graph and its representations

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More...

🕒 12 min read

Types of Graphs with Examples

A graph is a mathematical structure that represents relationships between objects by connecting a set of points. It is used to establish a pairwise relationship between elements in a given set. graphs are widely...

🕒 9 min read

Basic Properties of a Graph

A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph. The basi...

🕒 4 min read

Applications, Advantages and Disadvantages of Graph

Graph is a non-linear data structure that contains nodes (vertices) and edges. A graph is a collection of set of vertices and edges (formed by connecting two vertices). A graph is defined as $G = \{V, E\}$ where V i...

🕒 7 min read

Transpose graph

Transpose of a directed graph G is another directed graph on the same set of vertices with all of the edges reversed compared to the orientation of the corresponding edges in G . That is, if G contains an...

🕒 9 min read

Difference Between Graph and Tree

Graphs and trees are two fundamental data structures used in computer science to represent relationships between objects. While they share some similarities, they also have distinct differences th...

🕒 2 min read

BFS and DFS on Graph



Cycle in a Graph

